

# Entity Framework

## Introdução

O Entity Framework (EF) é um ORM (Object-Relational Mapper) utilizado para mapear objetos do domínio para estruturas relacionais em banco de dados.

Ele permite que a aplicação interaja com o banco utilizando código C#, abstraindo a necessidade de escrever queries SQL diretamente na maior parte dos casos.

*No contexto da LATAM Platform V2, o EF será o **padrão oficial** de acesso a dados para bancos relacionais, alinhado com a arquitetura baseada em **Clean Architecture** e **DDD**.*

## Como o Entity Framework funciona

O EF trabalha com alguns conceitos principais:

### DbContext

É a principal classe de acesso ao banco.

#### Responsabilidades:

- Gerenciar conexão
- Rastrear entidades (tracking)
- Executar queries
- Persistir alterações

#### Exemplo básico:

```
public class ApplicationDbContext : DbContext
{
    public DbSet<Transaction> Transactions { get; set; }
    public DbSet<User> Users { get; set; }
    public DbSet<Exception> Exceptions { get; set; }

    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(ApplicationDbContext).Assembly);
    }
}
```

### DbSet

Representa uma coleção de entidades no banco.

Exemplo:

- Transactions
- Users
- Exceptions

```
// Acessando um DbSet
var activeTransactions = await _context.Transactions
    .Where(t => t.Status == TransactionStatus.Active)
    .ToListAsync();
```

## Entidades

São classes que representam os dados do domínio.

Importante:

- Não devem depender de EF diretamente
- Devem representar o domínio, não o banco

Exemplo correto:

```
// Domain/Entities/Transaction.cs
public class Transaction
{
    public Guid Id { get; private set; }
    public string TransactionNumber { get; private set; }
    public decimal Amount { get; private set; }
    public TransactionStatus Status { get; private set; }
    public DateTime CreatedAt { get; private set; }

    // Construtor privado para EF
    private Transaction() { }

    // Factory method para criação
    public static Transaction Create(string transactionNumber, decimal amount)
    {
        return new Transaction
        {
            Id = Guid.NewGuid(),
            TransactionNumber = transactionNumber,
            Amount = amount,
            Status = TransactionStatus.Pending,
            CreatedAt = DateTime.UtcNow
        };
    }

    // Métodos de domínio
    public void Approve()
    {
        if (Status != TransactionStatus.Pending)
            throw new InvalidOperationException("Only pending transactions can be approved");

        Status = TransactionStatus.Approved;
    }
}
```

## Change Tracking

O EF rastreia alterações nas entidades automaticamente.

Isso permite:

- Atualizações automáticas
- Controle de estado (Added, Modified, Deleted)

Exemplo:

```
// EF rastreia a mudança automaticamente
var transaction = await _context.Transactions.FindAsync(id);
transaction.Approve(); // Altera o estado
await _context.SaveChangesAsync(); // Persiste a mudança
```

Quando desabilitar o tracking:

```
// Para queries de leitura (melhor performance)
var transactions = await _context.Transactions
    .AsNoTracking()
    .Where(t => t.CreatedAt > DateTime.UtcNow.AddDays(-30))
    .ToListAsync();
```

## LINQ

Permite escrever queries em C# que são convertidas para SQL.

```
// LINQ
var result = await _context.Users
    .Where(u => u.Country == "BR" && u.IsActive)
    .OrderByDescending(u => u.CreatedAt)
    .Select(u => new UserDto
    {
        Id = u.Id,
        Name = u.Name,
        Email = u.Email
    })
    .ToListAsync();

// SQL gerado pelo EF:
// SELECT u.Id, u.Name, u.Email
// FROM Users u
// WHERE u.Country = 'BR' AND u.IsActive = 1
// ORDER BY u.CreatedAt DESC
```

## Como usamos no projeto

No contexto da LATAM Platform V2:

| Aspecto     | Implementação                                            |
|-------------|----------------------------------------------------------|
| Uso         | Persistência relacional padrão                           |
| Arquitetura | Respeita separação de camadas (DDD + Clean Architecture) |
| Localização | Isolado na camada de <b>Infrastructure</b>               |
| Domínio     | EF não deve vazar para domínio ou interface              |

Estrutura de camadas:

```

LatamPlatform.Domain
├── Entities/
│   └── Transaction.cs
├── Repositories/
│   └── ITransactionRepository.cs
└──

LatamPlatform.Application
├── Services/
│   ├── ITransactionService.cs
│   └── TransactionService.cs
└──

LatamPlatform.Infrastructure
├── Persistence/
│   └── Repositories/
│       └── TransactionRepository.cs (implementa ITransactionRepository)
└──

```

Como implementar

## 1. Criar o DbContext

O DbContext representa o banco.

Boas práticas:

- Deve ser simples
- Não conter lógica de negócio
- Apenas configuração e acesso

Exemplo:

```

// Infrastructure/Persistence/ApplicationDbContext.cs
public class ApplicationDbContext : DbContext
{
    public DbSet<Transaction> Transactions { get; set; }
    public DbSet<User> Users { get; set; }

    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Aplicar todas as configurações do assembly
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(ApplicationDbContext).Assembly);
    }
}

```

## 2. Definir entidades

As entidades devem:

- Representar o domínio
- Ter identidade clara
- Não depender de EF

Evitar:

- atributos de infraestrutura no domínio
- acoplamento com banco

Exemplo correto:

```
// Domain/Entities/User.cs
public class User
{
    public Guid Id { get; private set; }
    public string Name { get; private set; }
    public string Email { get; private set; }
    public string Country { get; private set; }
    public bool IsActive { get; private set; }
    public DateTime CreatedAt { get; private set; }

    private User() { } // Para EF

    public static User Create(string name, string email, string country)
    {
        // Validações de domínio
        if (string.IsNullOrEmpty(email))
            throw new ArgumentException("Email is required");

        return new User
        {
            Id = Guid.NewGuid(),
            Name = name,
            Email = email,
            Country = country,
            IsActive = true,
            CreatedAt = DateTime.UtcNow
        };
    }

    public void Deactivate()
    {
        IsActive = false;
    }
}
```

### 3. Configurar mapeamentos

O mapeamento deve ser separado da entidade.

Recomendado:

- usar classes de configuração
- evitar configuração dentro do DbContext

Exemplo:

```
// Infrastructure/Persistence/Configurations/UserConfiguration.cs
public class UserConfiguration : IEntityTypeConfiguration<User>
{
    public void Configure(EntityTypeBuilder<User> builder)
    {
        builder.ToTable("Users");

        builder.HasKey(u => u.Id);

        builder.Property(u => u.Name)
            .IsRequired()
            .HasMaxLength(200);

        builder.Property(u => u.Email)
            .IsRequired()
            .HasMaxLength(320);

        builder.Property(u => u.Country)
            .IsRequired()
            .HasMaxLength(2);

        builder.HasIndex(u => u.Email)
            .IsUnique();

        builder.HasIndex(u => new { u.Country, u.IsActive });
    }
}
```

Exemplo com relacionamento:

```
// Infrastructure/Persistence/Configurations/TransactionConfiguration.cs
public class TransactionConfiguration : IEntityTypeConfiguration<Transaction>
{
    public void Configure(EntityTypeBuilder<Transaction> builder)
    {
        builder.ToTable("Transactions");

        builder.HasKey(t => t.Id);

        builder.Property(t => t.TransactionNumber)
            .IsRequired()
            .HasMaxLength(50);

        builder.Property(t => t.Amount)
            .HasColumnType("decimal(18,2)");

        builder.Property(t => t.Status)
            .HasConversion<string>(); // Enum como string

        // Relacionamento
        builder.HasOne(t => t.User)
            .WithMany(u => u.Transactions)
            .HasForeignKey(t => t.UserId)
            .OnDelete(DeleteBehavior.Restrict);

        builder.HasIndex(t => t.TransactionNumber)
            .IsUnique();
    }
}
```

## 4. Injeção de dependência

O DbContext deve ser registrado via DI.

Isso permite:

- controle de ciclo de vida
- testabilidade
- desacoplamento

Exemplo (Program.cs):

```
// Registrar DbContext
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection"),
        sqlOptions => sqlOptions.EnableRetryOnFailure(
            maxRetryCount: 3,
            maxRetryDelay: TimeSpan.FromSeconds(30),
            errorNumbersToAdd: null
        )
    );

// Registrar repositórios
builder.Services.AddScoped<ITransactionRepository, TransactionRepository>();
builder.Services.AddScoped<IUserRepository, UserRepository>();
```

Uso no serviço:

```
public class TransactionService
{
    private readonly ITransactionRepository _repository;

    public TransactionService(ITransactionRepository repository)
    {
        _repository = repository;
    }

    public async Task<Transaction> GetByIdAsync(Guid id)
    {
        return await _repository.GetByIdAsync(id);
    }
}
```

## Migrations

Migrations são usadas para **versionar alterações** no banco.

### Como funcionam

- Cada mudança no modelo gera uma migration
- EF gera scripts automaticamente
- Permite evolução incremental do schema

## Regras do projeto

| Regra                                          | Descrição                           |
|------------------------------------------------|-------------------------------------|
| Toda alteração estrutural deve virar migration | Nunca alterar banco manualmente     |
| Migrations devem estar no repositório          | Controle de versão obrigatório      |
| Não alterar migrations já aplicadas            | Criar nova migration para correções |

Nomear migrations de forma descritiva

Ex: AddTransactionStatusColumn

## Exemplo de fluxo

Alterar modelo

```
// Adicionar nova propriedade
public class Transaction
{
    // ... propriedades existentes
    public string Description { get; private set; } // NOVA
}
```

Gerar migration

```
dotnet ef migrations add AddDescriptionToTransaction --project Infrastructure --startup-project API
```

Revisar migration

```
// Migrations/20250328_AddDescriptionToTransaction.cs
public partial class AddDescriptionToTransaction : Migration
{
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.AddColumn<string>(
            name: "Description",
            table: "Transactions",
            type: "nvarchar(500)",
            maxLength: 500,
            nullable: true);
    }

    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropColumn(
            name: "Description",
            table: "Transactions");
    }
}
```

Aplicar migration



```
# Desenvolvimento
dotnet ef database update --project Infrastructure --startup-project API

# Produção (via CI/CD)
dotnet ef migrations bundle --output ./efbundle
./efbundle --connection "Server=prod;Database=LatamDB;..."
```

## Queries

EF deve ser o padrão para queries.

### Uso recomendado

- Queries simples e médias
- Operações transacionais
- Filtros e joins comuns

Exemplo de repositório:

```
public class TransactionRepository : ITransactionRepository
{
    private readonly ApplicationDbContext _context;

    public TransactionRepository(ApplicationDbContext context)
    {
        _context = context;
    }

    public async Task<Transaction> GetByIdAsync(Guid id)
    {
        return await _context.Transactions
            .Include(t => t.User)
            .FirstOrDefaultAsync(t => t.Id == id);
    }

    public async Task<IEnumerable<Transaction>> GetByCountryAsync(string country)
    {
        return await _context.Transactions
            .AsNoTracking()
            .Where(t => t.User.Country == country)
            .OrderByDescending(t => t.CreatedAt)
            .ToListAsync();
    }

    public async Task AddAsync(Transaction transaction)
    {
        await _context.Transactions.AddAsync(transaction);
        await _context.SaveChangesAsync();
    }
}
```

## Quando usar Raw SQL

- Queries muito complexas
- Performance crítica
- Limitações do EF

Exemplo:

```

public async Task<IEnumerable<TransactionSummary>> GetMonthlySummaryAsync(int year, int month)
{
    var sql = @"
        SELECT
            CAST(CreatedAt AS DATE) as Date,
            COUNT(*) as TotalTransactions,
            SUM(Amount) as TotalAmount
        FROM Transactions
        WHERE YEAR(CreatedAt) = {0} AND MONTH(CreatedAt) = {1}
        GROUP BY CAST(CreatedAt AS DATE)
        ORDER BY Date";

    return await _context.Database
        .SqlQueryRaw<TransactionSummary>(sql, year, month)
        .ToListAsync();
}

```

## Performance

EF abstrai SQL, mas **não elimina problemas de performance**.

### Boas práticas

| Prática                              | Implementação                                        |
|--------------------------------------|------------------------------------------------------|
| Evitar carregar dados desnecessários | Use <code>Select()</code> para projeções             |
| Preferir projeções                   | DTOs ao invés de entidades completas                 |
| Usar leitura otimizada               | <code>AsNoTracking()</code> quando não houver update |
| Evitar múltiplas queries             | Use <code>Include()</code> para eager loading        |

### Problemas comuns

- N+1 queries

```

var users = await _context.Users.ToListAsync();
foreach (var user in users)
{
    // Executa 1 query por usuário!
    var transactions = await _context.Transactions
        .Where(t => t.UserId == user.Id)
        .ToListAsync();
}

```

- Bom (1 query):

```

var users = await _context.Users
    .Include(u => u.Transactions)
    .ToListAsync();

```

- Melhor (projeção):

```
var users = await _context.Users
    .Select(u => new UserWithTransactionsDto
    {
        UserId = u.Id,
        UserName = u.Name,
        TransactionCount = u.Transactions.Count,
        TotalAmount = u.Transactions.Sum(t => t.Amount)
    })
    .ToListAsync();
```

## Multi-tenant

A plataforma utiliza isolamento por país (database per country)

### Implicações no EF

- O DbContext deve usar o banco correto por tenant
- Nenhuma operação deve ocorrer sem contexto de tenant
- Conexão deve ser resolvida dinamicamente

```

// Services/TenantService.cs
public class TenantService : ITenantService
{
    private readonly IHttpContextAccessor _httpContextAccessor;

    public string GetCurrentTenant()
    {
        // Obter tenant do header, claim, etc.
        return _httpContextAccessor.HttpContext?.User
            .FindFirst("country")?.Value ?? "BR";
    }
}

// Infrastructure/Persistence/TenantDbContext.cs
public class TenantDbContext : ApplicationDbContext
{
    private readonly ITenantService _tenantService;
    private readonly IConfiguration _configuration;

    public TenantDbContext(
        DbContextOptions<TenantDbContext> options,
        ITenantService tenantService,
        IConfiguration configuration)
        : base(options)
    {
        _tenantService = tenantService;
        _configuration = configuration;
    }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        if (!optionsBuilder.IsConfigured)
        {
            var tenant = _tenantService.GetCurrentTenant();
            var connectionString = _configuration
                .GetConnectionString($"LatamDB_{tenant}");

            optionsBuilder.UseSqlServer(connectionString);
        }
    }
}

// Program.cs
builder.Services.AddScoped<ITenantService, TenantService>();
builder.Services.AddScoped<ApplicationDbContext, TenantDbContext>();

```

## Boas práticas

- Usar EF como padrão para acesso a dados relacionais
- Manter DbContext simples e focado
- Separar configuração de entidades em classes IEntityTypeConfiguration
- Usar migrations sempre para mudanças estruturais
- Evitar lógica de negócio na camada de persistência
- Manter queries explícitas e legíveis
- Usar AsNoTracking() para queries de leitura
- Aplicar índices em colunas de filtro frequente
- Documentar queries complexas

```

/// <summary>
/// Busca transações ativas do último mês agrupadas por país.
/// Performance: ~200ms para 10k registros.
/// Índice requerido: IX_Transactions_CreatedAt_Status
/// </summary>
public async Task<IEnumerable<CountryTransactionSummary>> GetMonthlyByCountryAsync()
{
    return await _context.Transactions
        .AsNoTracking()
        .Where(t =>
            t.Status == TransactionStatus.Active &&
            t.CreatedAt >= DateTime.UtcNow.AddMonths(-1))
        .GroupBy(t => t.User.Country)
        .Select(g => new CountryTransactionSummary
        {
            Country = g.Key,
            TotalTransactions = g.Count(),
            TotalAmount = g.Sum(t => t.Amount)
        })
        .ToListAsync();
}

```

## O que evitar

- Expor entidades diretamente na API
- Misturar domínio com infraestrutura
- Usar EF de forma inconsistente entre projetos
- Criar queries genéricas sem necessidade
- Usar Raw SQL sem justificativa
- Ignorar impacto de performance
- Lazy loading em loops
- Modificar migrations já aplicadas
- Connection strings hardcoded